

GestorAlquiler — RentaCar

Documentación del Proyecto Final
Módulo: Programación | Curso 2024–2025

Alumna: Lucía Melero
IES Francisco Ayala

1. Descripción general del proyecto

GestorAlquiler es una aplicación de escritorio desarrollada en Java para gestionar una empresa de alquiler de coches. Es el proyecto final del módulo de Programación del curso 2024–2025.

La aplicación permite gestionar de forma completa los vehículos de la flota, los clientes y empleados registrados, y los alquileres activos. Además, incluye un conversor de divisas en tiempo real integrado con una API externa (ExchangeRate-API) que muestra el precio de cada vehículo en diferentes monedas.

El sistema distingue dos roles de usuario con permisos diferentes:

- Empleado: acceso completo a todos los módulos (Vehículos, Alquileres, Usuarios).
- Cliente: puede consultar vehículos disponibles, ver el conversor de divisas y crear sus propios alquileres.

2. Arquitectura y estructura del proyecto

2.1 Estructura de paquetes

```
src/  
  api/      → ExchangeRateService.java  
  db/       → ConexionDB.java  
  model/    → Usuario, Cliente, Empleado, Vehiculo, Alquiler  
  dao/      → Interfaces + implementaciones  
  dto/      → AlquilerDTO  
  view/     → Login.java, Registro.java, VentanaPrincipal.java  
  Main.java  
  lib/      → JARs externos (MySQL Connector)  
  sql/      → rentacar.sql  
  documentacion/
```

2.2 Arquitectura MVC

El proyecto sigue el patrón Modelo - Vista - Controlador:

- Model (model/): clases POJO que representan los datos. No contienen lógica SQL ni de interfaz.
- View (view/): ventanas Swing. No contienen ninguna línea de SQL. Se comunican con la base de datos exclusivamente a través de los DAOs.
- Controller (dao/): implementaciones DAO con toda la lógica de acceso a datos. Reciben objetos del modelo, ejecutan SQL y devuelven objetos del modelo.

2.3 Patrón DAO

Cada entidad tiene su interfaz (contrato) y su implementación concreta:

Interfaz	Implementación	Métodos principales
IUsuarioDAO	UsuarioDAOImpl	validar, registrar, listarTodos, actualizar, eliminar
IVehiculoDAO	VehiculoDAOImpl	insertar, actualizar, eliminar, listarTodos
IAlquilerDAO	AlquilerDAOImpl	insertar, actualizar, eliminar, listarTodos, buscarPorId

Buenas prácticas aplicadas en todos los DAOs:

- try-with-resources en todos los bloques JDBC para cerrar conexiones automáticamente.
- PreparedStatement siempre, sin concatenar SQL con valores del usuario (prevención de SQL injection).
- setAutoCommit(false) / commit() / rollback() en todas las operaciones que modifican varias tablas a la vez.

3. Modelo de base de datos

3.1 Descripción de las tablas

La base de datos se llama rentacar y sigue el patrón Joined Table Inheritance:

- usuarios: tabla raíz con los campos comunes a todos los usuarios (id, username, password, email, nombre, apellidos, dni, rol).
- clientes: tabla hija que amplía usuarios con telefono, direccion y carnet_conducir. La FK primaria usuario_id referencia a usuarios(id) con ON DELETE CASCADE.
- empleados: tabla hija que amplía usuarios con salario y fecha_alta. Misma estructura de FK.
- vehiculos: entidad principal del dominio con matricula, marca, modelo, anio, categoria, precio_dia y disponible.
- alquileres: tabla de relación N:M entre clientes y vehículos. Contiene cliente_id, vehiculo_id, empleado_id (nullable), fecha_inicio, fecha_fin, precio_total y estado.

3.2 Diagrama de relaciones

usuarios (id PK, username, password, email, nombre, apellidos, dni, rol)

├── clientes (usuario_id PK/FK, telefono, direccion, carnet_conducir)

└── empleados (usuario_id PK/FK, salario, fecha_alta)

vehiculos (id PK, matricula, marca, modelo, anio, categoria, precio_dia, disponible)

alquileres (id PK, cliente_id FK, vehiculo_id FK, empleado_id FK nullable,

fecha_inicio, fecha_fin, precio_total, estado)

3.3 Herencia — Joined Table Inheritance

La herencia de usuarios en la base de datos se implementa con Joined Table Inheritance: la tabla usuarios almacena los campos comunes y cada tabla hija (clientes, empleados) tiene su propio registro vinculado por una FK primaria que es a la vez clave foránea de usuarios(id). Esto se refleja en el modelo Java con la clase base Usuario y las subclases Cliente y Empleado que la extienden con extends.

4. Instrucciones de instalación y ejecución

4.1 Requisitos previos

- Java 17 o superior
- MySQL 8
- Visual Studio Code con Extension Pack for Java

4.2 Pasos

- Clona el repositorio o descomprime el ZIP del proyecto.
- Abre MySQL y ejecuta el script `sql/rentacar.sql` para crear la base de datos con los datos de prueba.
- Abre `src/db/ConexionDB.java` y ajusta USER y PASS con tus credenciales de MySQL si son diferentes a las por defecto.
- Abre la carpeta del proyecto en VS Code.
- Pulsa F5 o ejecuta `Main.java` para arrancar la aplicación.

4.3 Usuarios de prueba

Usuario	Contraseña	Rol	Nombre
admin	1234	empleado	Carlos López
laura	1234	empleado	Laura Sánchez
maria	1234	cliente	María García
juan	1234	cliente	Juan Martínez

5. Repositorio GitHub

El proyecto está publicado en GitHub con commits regulares a lo largo del desarrollo que documentan la evolución del proyecto:

https://github.com/luciamelerof/gestoralquiler_luciamelero

6. Extensiones implementadas

6.1 Conversión de divisas — ExchangeRate-API

En el módulo de Vehículos hay un panel lateral que convierte el precio diario del vehículo seleccionado a otras monedas: USD, GBP, JPY, CHF y MXN.

- API utilizada: ExchangeRate-API (<https://www.exchangerate-api.com/>), capa gratuita.
- Integración: `src/api/ExchangeRateService.java` realiza una petición HTTP con `HttpURLConnection`, parsea la respuesta JSON manualmente y devuelve el tipo de cambio.
- La llamada se ejecuta en un hilo separado (`new Thread`) para no bloquear la interfaz mientras espera la respuesta. El resultado se actualiza con `SwingUtilities.invokeLater` para respetar el hilo de Swing.

7. WakaTime

El tiempo de desarrollo ha sido registrado con WakaTime bajo el nombre de proyecto ProyectoFinal-Programacion durante todas las sesiones de trabajo.

Enlace al dashboard público / captura del informe:

<https://wakatime.com/@d67ce14a-901e-439a-b990-c47ccd5ace0a/projects/ddewvrtzgs>